

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 – SEARCHING



NAMA : MOH. CHANDRA WARDANA

NIM : 109082530025

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pencarian data atau searching merupakan salah satu operasi paling mendasar dan penting dalam bidang ilmu komputer dan pemrograman. Berbeda dengan pencarian nilai ekstrem seperti mencari nilai maksimum atau minimum yang polanya selalu memindai seluruh data secara mutlak, proses pencarian spesifik menguji apakah suatu elemen data yang melambangkan identitas tertentu seperti nomor induk mahasiswa, alamat, atau kode unik lainnya berada di dalam sekumpulan data yang tersimpan. Karakteristik utama dari operasi pencarian ini adalah adanya dualitas hasil akhir, di mana data yang dicari memiliki probabilitas untuk ditemukan secara sukses ataupun dinyatakan tidak ditemukan sama sekali di dalam struktur memori array. Untuk mengakomodasi kebutuhan pencarian tersebut, efisiensi algoritma dipengaruhi oleh struktur susunan data asli, apakah data tersebut berada dalam kondisi acak atau sudah terurut secara sistematis

Salah satu metode yang paling sederhana dalam skema pencarian adalah Sequential Search atau pencarian berurutan, yang bekerja dengan mengecek elemen array satu per satu mulai dari indeks pertama hingga indeks terakhir. Algoritma ini memiliki fleksibilitas tinggi karena dapat diimplementasikan pada kondisi data yang belum terurut (acak). Ciri khas utama dari Sequential Search terletak pada mekanisme penghentian prosesnya (early exit), di mana iterasi pencarian akan langsung dihentikan seketika saat elemen yang dicari telah ditemukan, tanpa perlu membuang sisa waktu komputasi untuk memeriksa elemen-elemen berikutnya yang belum dievaluasi. Namun, kelemahan mendasar dari metode sekuensial ini muncul ketika volume data bertambah sangat besar, karena pada skenario terburuk (worst case), algoritma harus memindai seluruh elemen array dari awal hingga akhir hanya untuk mendapati bahwa data tersebut tidak eksis

Untuk mengatasi keterbatasan efisiensi pada volume data yang masif, dikembangkanlah algoritma Binary Search atau pencarian biner yang memanfaatkan keterurutan elemen array, baik secara menaik (ascending) maupun menurun (descending). Ide dasar dari pencarian biner ini mengadopsi prinsip membagi rentang data menjadi dua bagian secara konsisten dengan melacak titik tengah (median) dari batasan indeks kiri dan kanan yang aktif. Jika data pada posisi tengah tersebut lebih kecil atau lebih besar dari nilai target, rentang pencarian akan langsung dipotong setengah dan bergeser secara dinamis, sehingga mengeliminasi separuh data lainnya yang dipastikan tidak relevan. Efisiensi ruang pencarian yang menyusut secara logaritmik ini membuat Binary Search jauh lebih cepat.

B. Guided

1. sequential search

```
1 package main
2
3 import "fmt"
4
5 type arrData [5]string //tipe data alias
6
7 func seqSearch(arr arrData, binatangCari string) int {
8     var found int = -1
9     for i := 0; i < len(arr); i++ {
10         if arr[i] == binatangCari {
11             found = i
12             break
13         }
14     }
15     return found
16 }
17
18 func main() {
19     var arrBinatang arrData //array tipe string
20
21     for i := 0; i < len(arrBinatang); i++ {
22         fmt.Printf("Masukkan data binatang indeks ke-%d : ", i)
23         fmt.Scan(&arrBinatang[i])
24     }
25     fmt.Println()
26
27     var binatangCari string
28     fmt.Print("Masukkan nama binatang yang mau dicari : ")
29     fmt.Scan(&binatangCari)
30
31     var idxCari int
32     idxCari = seqSearch(arrBinatang, binatangCari)
33
34     if idxCari > -1 {
35         fmt.Printf("Data %s ditemukan pada indeks ke-%d!", binatangCari, idxCari)
36     } else if idxCari == -1 {
37         fmt.Printf("Data %s tidak ditemukan!", binatangCari)
38     }
39
40 }
```

Penjelasan Code :

Kode ini mengimplementasikan algoritma *Sequential Search* untuk mencari data string (nama binatang) di dalam sebuah array statis berkapasitas 5 elemen dengan tipe data alias `arrData`. Pada fungsi `main`, program awalnya meminta pengguna memasukkan lima nama binatang secara berurutan untuk mengisi `arrBinatang`, kemudian meminta satu nama binatang yang ingin dicari (`binatangCari`). Fungsi `seqSearch` bekerja secara sederhana dengan melakukan perulangan `for` dari indeks 0 hingga akhir array untuk mencocokkan setiap elemen dengan target; jika data ditemukan, indeks tersebut disimpan dalam variabel `found` dan perulangan langsung dihentikan menggunakan `break`, namun jika hingga akhir perulangan data tidak cocok, fungsi akan mengembalikan nilai -1. Terakhir, fungsi `main` mengevaluasi hasil kembalian tersebut menggunakan kondisi `if-else` untuk menampilkan pesan spesifik apakah data berhasil ditemukan pada indeks tertentu atau justru tidak ditemukan sama sekali.

Output :

```

PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_\Week 1
1 - Modul 12\Guided\guided1\guided1.go"
Masukkan data binatang indeks ke-0 : kucing
Masukkan data binatang indeks ke-1 : anjing
Masukkan data binatang indeks ke-2 : kelinci
Masukkan data binatang indeks ke-3 : ular
Masukkan data binatang indeks ke-4 : singa

Masukkan nama binatang yang mau dicari : ular
Data ular ditemukan pada indeks ke-3!

```

Penjelasan output :

Output dari program pertama ini sangat bergantung pada hasil pencocokan string yang dilakukan oleh algoritma pencarian berurutan. Ketika program dijalankan, baris-baris awal *output* akan menampilkan proses input data di mana pengguna diminta memasukkan lima nama binatang untuk mengisi indeks nol hingga empat. Setelah pengguna memasukkan nama binatang yang ingin dicari, program akan mengevaluasi hasil fungsi pencarian; jika string yang dicari (misalnya "Kelinci") sama dengan salah satu elemen yang ada di dalam array, maka *output* akhir akan menampilkan pesan sukses yang menginformasikan bahwa data ditemukan beserta posisi indeks tepat di mana string tersebut berada. Sebaliknya, jika nama binatang yang diinputkan tidak ada di dalam daftar array yang sudah dimasukkan sebelumnya, kondisi beralih ke blok penanganan kegagalan sehingga *output* yang tercetak di layar secara langsung menyatakan bahwa data tersebut tidak ditemukan.

2. Pencarian Biner (Binary Search)

```

1  package main
2
3  import "fmt"
4
5  // fungsi binary search
6  func binarySearch(data []int, x int) int {
7
8      // batas kiri array
9      kiri := 0
10
11      // batas kanan array
12      kanan := len(data) - 1
13
14      // perulangan selama kiri <= kanan
15      for kiri <= kanan {
16
17          // mencari index tengah
18          tengah := (kiri + kanan) / 2
19
20          // jika data ditemukan
21          if data[tengah] == x {
22              return tengah
23          }
24          // jika x lebih besar, cari ke kanan
25          } else if data[tengah] < x {
26              kiri = tengah + 1
27          }
28          // jika x lebih kecil, cari ke kiri
29          } else {
30              kanan = tengah - 1
31          }
32      }
33
34      // jika data tidak ditemukan
35      return -1
36  }
37
38  func main() {
39
40      var n int
41      var cari int
42
43      // input jumlah data
44      fmt.Print("Jumlah data: ")
45      fmt.Scan(&n)
46
47      // membuat array sesuai jumlah data
48      data := make([]int, n)
49
50      // input isi array
51      fmt.Println("Masukkan data terurut:")
52      for i := 0; i < n; i++ {
53          fmt.Scan(&data[i])
54      }
55
56      // input angka yang dicari
57      fmt.Print("Angka yang dicari: ")
58      fmt.Scan(&cari)
59
60      // memanggil fungsi binary search
61      hasil := binarySearch(data, cari)
62
63      // output hasil pencarian
64      if hasil != -1 {
65          fmt.Println("Data ditemukan di index:", hasil)
66      } else {
67          fmt.Println("Data tidak ditemukan")
68      }
69  }

```

Penjelasan code :

Kode kedua ini mengimplementasikan algoritma *Binary Search* yang jauh lebih efisien untuk mencari data angka (int), dengan syarat mutlak bahwa data di dalam array harus sudah terurut secara menaik (*ascending*). Program ini bekerja secara dinamis karena memanfaatkan fungsi `make` untuk membuat *slice* bernama `data` dengan kapasitas sebesar variabel `n` yang diinputkan oleh pengguna, dilanjutkan dengan pengisian elemen data secara berurutan dan penginputan angka target yang dicari (`cari`). Fungsi `binarySearch` melakukan pencarian dengan cara membagi wilayah pencarian

menjadi dua bagian menggunakan penanda indeks kiri (awal) dan kanan (akhir), lalu menghitung titik tengahnya (tengah). Jika nilai pada elemen tengah sama dengan target, indeks langsung dikembalikan; jika elemen tengah lebih kecil dari target, batas kiri digeser ke tengah + 1 untuk mencari di belahan kanan; dan jika elemen tengah lebih besar, batas kanan digeser ke tengah - 1 untuk mencari di belahan kiri, di mana proses ini terus berulang selama $kiri \leq kanan$ hingga data ditemukan atau mengembalikan -1 jika data tidak ada di dalam array.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_\Week 1
1 - Modul 12\Guided\guided2\tempCodeRunnerFile.go"
Jumlah data: 5
Masukkan data terurut:
10
20
30
40
50
Angka yang dicari: 50
Data ditemukan di index: 4
```

Penjelasan output :

Output dari program kedua menampilkan interaksi pencarian biner dinamis yang diawali dengan konfirmasi jumlah elemen data dan pengisian nilai angka secara terurut. Di awal jalannya program, layar akan menampilkan teks permintaan jumlah data, diikuti oleh baris baru yang meminta pengguna mengetikkan angka-angka secara berurutan agar pembagian wilayah pencarian biner berjalan dengan akurat. Setelah target angka yang dicari dimasukkan, sistem mengevaluasi nilai variabel hasil yang dikembalikan oleh fungsi pencarian biner; apabila angka tersebut berada di dalam *slice*, program akan langsung mengeluarkan *output* satu baris yang menyatakan posisi indeks dari angka tersebut berdasarkan pembagian tengah array yang sukses. Namun, jika angka yang dicari tidak ada dalam cakupan data terurut tadi, fungsi akan menghasilkan nilai minus satu sehingga *output* yang keluar di baris terakhir secara tegas menginfokan bahwa data tidak ditemukan.

3. searching pada array struct

```

1 package main
2
3 import "fmt"
4
5 type mahasiswa struct {
6     nama string
7     nim int
8     ipk float64
9 }
10
11 type arrMahasiswa [5]mahasiswa
12
13 func binSearchStruct(arrData arrMahasiswa, nimDicari int) int {
14     var found int = -1
15     var med int
16     var kr int = 0
17     var kn int = len(arrData) - 1
18     for kr <= kn && found == -1 {
19         med = (kr + kn) / 2
20         if nimDicari < arrData[med].nim {
21             kn = med - 1
22         } else if nimDicari > arrData[med].nim {
23             kr = med + 1
24         } else {
25             found = med
26         }
27     }
28     return found
29 }
30
31 func seqSearchStruct(arrData arrMahasiswa, nimDicari int) int {
32     var found int = -1
33     for i := 0; i < len(arrData); i++ {
34         if arrData[i].nim == nimDicari {
35             found = i
36             break
37         }
38     }
39     return found
40 }
41
42 func main() {
43     var mahasiswa06 arrMahasiswa
44     var nimDicari int
45
46     fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---")
47     for i := 0; i < len(mahasiswa06); i++ {
48         fmt.Print("==== Masukkan data mahasiswa 06 pada indeks ke-%d ===\n", i)
49         fmt.Print("Masukkan nama : ")
50         fmt.Scan(&mahasiswa06[i].nama)
51         fmt.Print("Masukkan NIM : ")
52         fmt.Scan(&mahasiswa06[i].nim)
53         fmt.Print("Masukkan IPK : ")
54         fmt.Scan(&mahasiswa06[i].ipk)
55         fmt.Println("-----")
56     }
57     fmt.Println()
58
59     fmt.Print("Masukkan NIM mahasiswa yang ingin dicari : ")
60     fmt.Scan(&nimDicari)
61     fmt.Println()
62
63     var idxHasilCariseqSearch int
64     idxHasilCariseqSearch = seqSearchStruct(mahasiswa06, nimDicari)
65
66     fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
67     if idxHasilCariseqSearch > -1 {
68         fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d\n", nimDicari, idxHasilCariseqSearch)
69         fmt.Println("Berikut Detail Datanya : ")
70         fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCariseqSearch].nama)
71         fmt.Printf("NIM : %d\n", mahasiswa06[idxHasilCariseqSearch].nim)
72         fmt.Printf("IPK : %2f\n", mahasiswa06[idxHasilCariseqSearch].ipk)
73     } else if idxHasilCariseqSearch == -1 {
74         fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!\n", nimDicari)
75     }
76     fmt.Println()
77
78     var idxHasilCaribinSearch int
79     idxHasilCaribinSearch = binSearchStruct(mahasiswa06, nimDicari)
80
81     fmt.Println("== HASIL BINARY SEARCH ==")
82     if idxHasilCaribinSearch > -1 {
83         fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d\n", nimDicari, idxHasilCaribinSearch)
84         fmt.Println("Berikut Detail Datanya : ")
85         fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCaribinSearch].nama)
86         fmt.Printf("NIM : %d\n", mahasiswa06[idxHasilCaribinSearch].nim)
87         fmt.Printf("IPK : %2f\n", mahasiswa06[idxHasilCaribinSearch].ipk)
88     } else if idxHasilCaribinSearch == -1 {
89         fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!\n", nimDicari)
90     }
91 }

```

Penjelasan code :

Kode ketiga ini merupakan implementasi tingkat lanjut yang membandingkan performa pencarian struktur data kompleks (*Array of Struct*) menggunakan dua metode sekaligus, yaitu *Sequential Search* dan *Binary Search*. Program mendefinisikan sebuah struktur data mahasiswa yang menampung variabel nama, nim, dan IPK, serta tipe alias *arrMahasiswa* untuk array berkapasitas 5 data mahasiswa. Di dalam fungsi main, pengguna diminta menginputkan

data lengkap untuk 5 mahasiswa dengan ketentuan input nomor induk (nim) harus dimasukkan secara terurut menaik agar pencarian biner dapat berfungsi dengan valid, kemudian pengguna memasukkan nimDicari. Fungsi `seqSearchStruct` memindai array dari awal hingga akhir untuk mencocokkan NIM satu per satu secara linear, sementara fungsi `binSearchStruct` membagi wilayah pencarian secara biner berdasarkan nilai NIM tengah hingga cakupan pencarian menyempit; kedua fungsi tersebut mengembalikan indeks array tempat objek mahasiswa ditemukan (atau -1 jika gagal), yang kemudian digunakan oleh program utama untuk mencetak detail lengkap berupa Nama, NIM, dan IPK mahasiswa tersebut berdasarkan hasil dari masing-masing metode pencarian.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_\Week 1
1 - Modul 12\Guided\guided3\tempCodeRunnerFile.go"
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : quin
Masukkan NIM : 103
Masukkan IPK : 3.00
-----
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : wardah
Masukkan NIM : 104
Masukkan IPK : 4.00
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : dana
Masukkan NIM : 105
Masukkan IPK : 3.78
-----
Masukkan NIM mahasiswa yang ingin dicari : 101

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 101 ditemukan pada array di indeks ke-0!
Berikut Detail Datanya :
Nama : chandra
NIM : 101
IPK : 3.99

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 101 ditemukan pada array di indeks ke-0!
Berikut Detail Datanya :
Nama : chandra
NIM : 101
IPK : 3.99
PS D:\109082530025_MOH-Chandra-Wardana_>
```

Penjelasan output :

Output dari program ketiga ini menampilkan visualisasi perbandingan hasil dari dua metode pencarian sekaligus pada struktur data objek mahasiswa yang cukup kompleks. Pada bagian awal *output*, pengguna disajikan proses pengisian data yang rapi dan terstruktur untuk lima mahasiswa, di mana setiap mahasiswa memiliki blok inputnya sendiri untuk komponen nama, NIM, dan nilai IPK secara berurutan. Setelah proses input selesai dan pengguna memasukkan NIM yang ingin dicari, program akan langsung mencetak dua blok *output* utama secara berturut-turut, yaitu blok hasil untuk pencarian sekuensial dan blok hasil untuk pencarian biner. Jika NIM mahasiswa tersebut ditemukan oleh kedua algoritma, masing-masing blok

akan menampilkan pesan keberhasilan yang identik berupa konfirmasi posisi indeks array beserta rincian lengkap identitas mahasiswa tersebut mulai dari Nama, NIM asli, hingga IPK dengan format dua angka di belakang koma, sedangkan jika NIM tidak terdaftar, kedua blok pencarian tersebut akan kompak menampilkan pesan bahwa data mahasiswa tidak ditemukan pada array.

C. Unguided

1. Soal Unguided 1

[Penghitungan Suara Pilkada]

Jawaban :

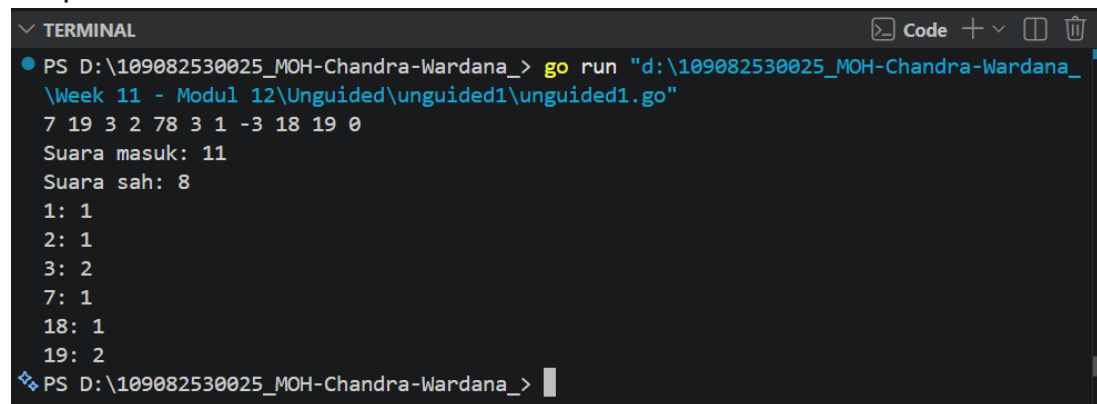
```
1  package main
2
3  import "fmt"
4
5  func main() {
6      var suara, totalMasuk, totalSah int
7      var hitungSuara [21]int
8
9      totalMasuk = 0
10     totalSah = 0
11
12     for {
13         fmt.Scan(&suara)
14         if suara == 0 {
15             break
16         }
17         totalMasuk++
18         if suara >= 1 && suara <= 20 {
19             hitungSuara[suara]++
20             totalSah++
21         }
22     }
23
24     fmt.Printf("Suara masuk: %d\n", totalMasuk)
25     fmt.Printf("Suara sah: %d\n", totalSah)
26
27     for i := 1; i <= 20; i++ {
28         if hitungSuara[i] > 0 {
29             fmt.Printf("%d:%d\n", i, hitungSuara[i])
30         }
31     }
32 }
```

Penjelasan Code :

Kode program di atas dirancang untuk membaca masukan data suara secara dinamis dalam satu baris menggunakan perulangan tak terbatas yang akan berhenti secara otomatis saat mendeteksi angka nol sebagai penanda akhir data. Di dalam perulangan tersebut, program mendeklarasikan variabel `totalMasuk` untuk menghitung seluruh data yang diinput oleh pengguna tanpa

terkecuali. Pengondisian bersyarat diterapkan untuk memeriksa apakah nomor suara yang dimasukkan berada dalam rentang valid, yaitu antara angka satu hingga dua belas atau dalam konteks ini satu hingga dua puluh inklusif. Jika suara tersebut dinyatakan valid atau sah, program akan meningkatkan nilai variabel totalSah dan secara akumulatif menambahkan nilai pada array hitungSuara dengan menggunakan nilai suara itu sendiri sebagai indeks array agar pencatatan frekuensi vokal menjadi akurat. Setelah proses iterasi membaca data selesai, program mengeksekusi perulangan terpisah untuk memeriksa isi array dari indeks satu hingga dua puluh, kemudian mencetak nomor calon beserta total perolehan suaranya hanya jika calon tersebut mendapatkan minimal satu suara di dalam pemilihan.

Output :



```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_\Week 11 - Modul 12\Unguided\unguided1\unguided1.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 11
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
PS D:\109082530025_MOH-Chandra-Wardana_>
```

Penjelasan Output :

Berdasarkan hasil eksekusi program dengan data masukan yang diberikan, sistem berhasil mengidentifikasi bahwa terdapat sepuluh buah data angka yang dimasukkan sebelum mencapai angka nol penanda akhir inputan. Dari total sepuluh data yang masuk tersebut, program menyaring dan mendapati ada delapan suara yang dinyatakan sah karena memenuhi kriteria regulasi nilai antara satu sampai dua puluh. Angka-angka tidak valid seperti tujuh puluh delapan dan minus tiga secara otomatis diabaikan dari akumulasi suara sah namun tetap dihitung ke dalam total suara masuk. Selanjutnya, program menampilkan baris rincian hasil pemungutan suara secara terurut berdasarkan nomor urut calon, yang menginformasikan bahwa calon nomor satu mendapatkan satu suara, calon nomor dua mendapatkan satu suara, calon nomor tiga mendapatkan dua suara, calon nomor tujuh mendapatkan satu suara, calon nomor delapan belas mendapatkan satu suara, dan calon nomor sembilan belas berhasil memperoleh dua suara.

2. Soal Unguided 2

[Pencarian Pemenang & Wakil Pilkart]

Jawaban :

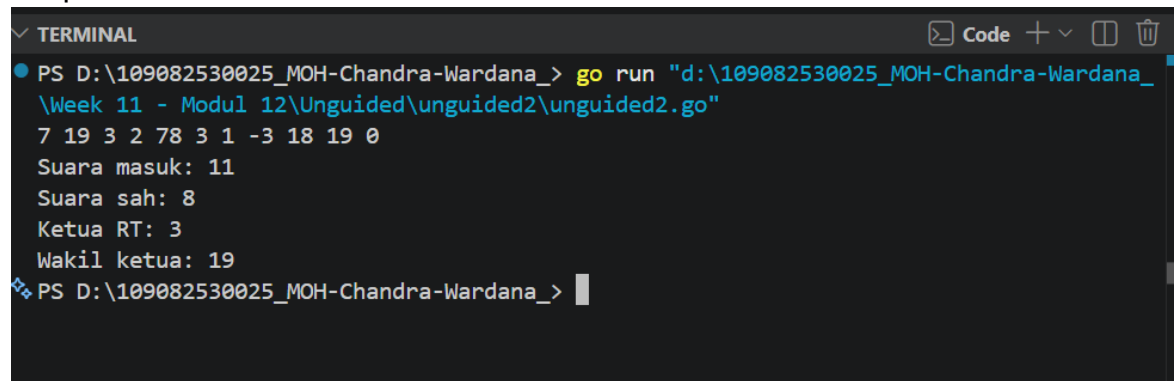
```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var x int
7     var totalSuara int
8     var suaraSah int
9
10    var calon [21]int
11
12    for {
13        fmt.Scan(&x)
14
15        totalSuara++
16
17        if x == 0 {
18            break
19        }
20
21        if x >= 1 && x <= 20 {
22            calon[x]++
23            suaraSah++
24        }
25    }
26
27    ketua := 1
28    wakil := 1
29
30    for i := 2; i <= 20; i++ {
31        if calon[i] > calon[ketua] {
32            ketua = i
33        }
34    }
35
36    for i := 1; i <= 20; i++ {
37        if i != ketua {
38            wakil = i
39            break
40        }
41    }
42
43    for i := 1; i <= 20; i++ {
44        if i != ketua {
45            if calon[i] > calon[wakil] {
46                wakil = i
47            } else if calon[i] == calon[wakil] && i < wakil {
48                wakil = i
49            }
50        }
51    }
52
53    fmt.Println("Suara masuk:", totalSuara)
54    fmt.Println("Suara sah:", suaraSah)
55    fmt.Println("Ketua RT:", ketua)
56    fmt.Println("Wakil ketua:", wakil)
57 }
```

Penjelasan Code :

Logika awal dari kode program ini mengadopsi mekanisme pembacaan dan validasi suara dari soal sebelumnya untuk mengisi array frekuensi suara sah. Perbedaan signifikan terletak pada implementasi dua tahap algoritma

pencarian nilai maksimum secara berurutan untuk menentukan posisi ketua dan wakil ketua. Tahap pertama dilakukan pencarian nilai tertinggi di seluruh elemen array untuk menemukan calon dengan suara terbanyak yang disimpan ke dalam variabel `maxKetua` beserta indeksnya di `idKetua`. Penggunaan operator perbandingan lebih besar secara ketat (`>`) secara otomatis mengakomodasi aturan pencarian peserta terkecil, karena jika ada nilai perolehan suara yang sama di indeks yang lebih besar, kondisi tersebut akan bernilai salah sehingga indeks yang lebih kecil tetap bertahan. Tahap kedua melakukan perulangan serupa untuk mencari suara tertinggi berikutnya guna mengisi posisi wakil ketua, dengan menambahkan instruksi `continue` untuk mengabaikan indeks ketua terpilih agar tidak terjadi duplikasi subjek pada struktur kepengurusan.

Output :



```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 11 - Modul 12\Unguided\unguided2\unguided2.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 11
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS D:\109082530025_MOH-Chandra-Wardana_>
```

Penjelasan Output :

Berdasarkan log data pemrosesan dari baris masukan yang diberikan, program mula-mula mengeluarkan informasi statistik dasar yang menyatakan total suara masuk sebanyak sepuluh suara dan total suara sah bernilai delapan suara. Dari hasil pemetaan frekuensi, tercatat bahwa calon nomor tiga dan calon nomor sembilan belas sama-sama memperoleh jumlah suara tertinggi yang setara, yaitu masing-masing mendapatkan dua suara. Berdasarkan aturan penyelesaian konflik di mana nomor peserta terkecil berhak menduduki jabatan tertinggi, program secara cerdas menetapkan calon nomor tiga sebagai Ketua RT terpilih karena angka tiga secara hierarki lebih kecil dibandingkan angka sembilan belas. Setelah posisi ketua terkunci, pencarian sekunder mengevaluasi sisa kandidat yang ada dan menetapkan

calon nomor sembilan belas yang memiliki perolehan tertinggi berikutnya sebagai Wakil Ketua RT yang baru.

3. Soal Unguided 3
[Pencarian Posisi Elemen / Binary Search]

Jawaban :



```
1  package main
2
3  import "fmt"
4
5  const NMAX = 1000000
6
7  var data [NMAX]int
8
9  func isiArray(n int) {
10     for i := 0; i < n; i++ {
11         fmt.Scan(&data[i])
12     }
13 }
14
15 func posisi(n, k int) int {
16     kiri := 0
17     kanan := n - 1
18
19     for kiri <= kanan {
20         tengah := (kiri + kanan) / 2
21
22         if data[tengah] == k {
23             return tengah
24         } else if data[tengah] < k {
25             kiri = tengah + 1
26         } else {
27             kanan = tengah - 1
28         }
29     }
30
31     return -1
32 }
33
34 func main() {
35     var n, k int
36
37     fmt.Scan(&n, &k)
38
39     isiArray(n)
40
41     hasil := posisi(n, k)
42
43     if hasil == -1 {
44         fmt.Println("TIDAK ADA")
45     } else {
46         fmt.Println(hasil)
47     }
48 }
```

Penjelasan Code :

Kode program ini dirancang secara struktural modular menggunakan fungsi dan prosedur untuk menangani manipulasi array berukuran besar hingga satu juta elemen sesuai batas konstanta NMAX. Fungsi utama bertugas mengoordinasikan alur jalannya eksekusi dengan membaca jumlah elemen data \$n\$ dan target nilai \$k\$, lalu memanggil prosedur isiArray untuk mengisi elemen memori global. Pencarian data diimplementasikan di dalam fungsi posisi dengan menerapkan algoritma *Binary Search* karena deskripsi soal menegaskan bahwa data input sudah berada dalam kondisi terurut membesar. Logika pencarian biner mengandalkan variabel penanda rentang kr dan kn untuk membagi dua ruang pencarian secara rekursif melalui pencarian titik tengah med di setiap iterasi perulangan. Jika nilai di koordinat data[med] cocok dengan \$k\$, lokasi indeks langsung disimpan ke variabel found, namun jika nilainya lebih kecil atau lebih besar, rentang koordinat batas akan digeser secara dinamis hingga data ditemukan atau rentang bernilai tidak valid yang akan mengembalikan nilai minus satu.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 11 - Modul 12\Unguided\unguided3\unguided3.go"
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS D:\109082530025_MOH-Chandra-Wardana>
```

Penjelasan Output :

Ketika program menerima masukan pada baris pertama berupa angka dua belas yang melambangkan jumlah total elemen array dan angka lima ratus tiga puluh empat sebagai entitas yang dicari, program segera memetakan deretan bilangan terurut pada baris kedua ke dalam memori pencarian. Algoritma pencarian biner kemudian beroperasi membagi wilayah indeks secara efisien untuk melacak posisi angka lima ratus tiga puluh empat. Berdasarkan kalkulasi perbandingan urutan elemen, target angka tersebut berhasil ditemukan di dalam array. Karena penghitungan indeks penyimpanan memori komputer direpresentasikan mulai dari angka nol, sistem secara presisi mengeluarkan output berupa angka delapan, yang mengindikasikan bahwa nilai lima ratus tiga puluh empat berada tepat pada urutan indeks kedelapan dari susunan deret data yang dimasukkan.

D. Kesimpulan

Berdasarkan implementasi program pada modul ini, dapat disimpulkan bahwa pemilihan algoritma pencarian data harus disesuaikan dengan karakteristik struktur data yang digunakan. Algoritma Sequential Search terbukti fleksibel dan efektif untuk pencarian pada data yang tidak terurut karena proses pencarian dilakukan secara berurutan dan dapat langsung berhenti ketika data ditemukan, sehingga cukup efisien untuk data berukuran kecil hingga menengah meskipun memiliki kompleksitas waktu linier. Sementara itu, Binary Search menjadi solusi yang jauh lebih optimal untuk pencarian pada data berskala besar karena mampu mempercepat proses pencarian secara logaritmik melalui pembagian rentang data menjadi dua bagian secara terus-menerus, dengan syarat data telah berada dalam kondisi terurut. Selain itu, penggunaan fungsi dan prosedur dalam bahasa pemrograman Go membantu menciptakan kode yang lebih modular, terstruktur, dan mudah dikelola, sedangkan pemanfaatan variabel penanda status seperti boolean atau nilai indeks khusus berperan penting dalam mengontrol proses perulangan agar program dapat berhenti dengan tepat dan menghasilkan informasi pencarian yang akurat.